

DEVELOPER DOCUMENTATION

AI Receptionist Platform

Architecture, data model, API reference and voice-integration walkthrough for the multi-tenant AI receptionist SaaS.

ASP.NET Core 8 · Dapper · SQL Server (AiDB) · React 18 + Vite
Version 1.0 · Internal engineering document

Contents

01 System overview

02 Technology stack

03 Repository layout

04 Multi-tenancy model

05 Database schema

06 Authentication & authorization

07 Request pipeline

08 Internal API reference

09 Prompt architecture

10 Industry templates

11 Voice provider integration

12 Live-call tool endpoints

13 Webhooks & call reconciliation

14 Background workers

15 Security model

16 Correctness & edge cases

17 Frontend architecture

18 Configuration reference

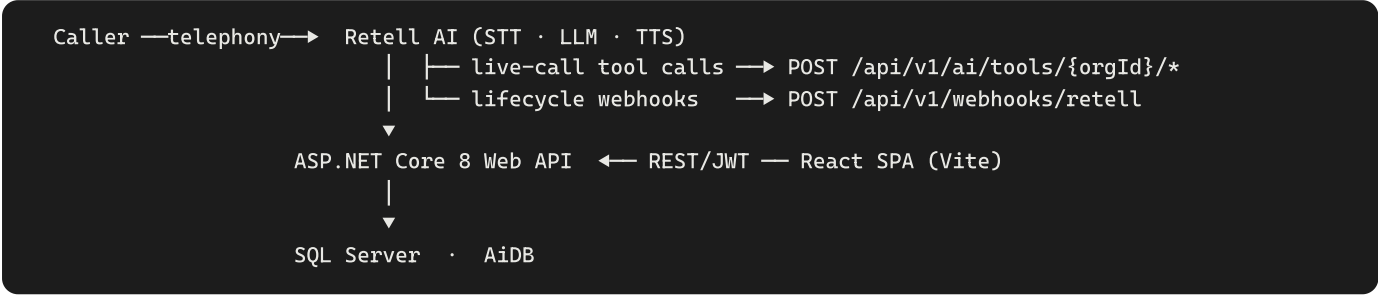
19 Local setup & operations

20 Known gaps & roadmap

01 System overview

A multi-tenant SaaS that answers inbound phone calls with an AI voice agent and gives each tenant a dashboard for appointments, customers, catalogue, knowledge and analytics. The platform is industry-independent: behaviour adapts by configuration, not by code changes.

Runtime topology



Design principles

- **The database is the source of truth.** The AI never answers from memory for prices, availability or bookings — it calls back into this API, which reads AiDB.
- **Tenant isolation is structural.** Every tenant-owned table carries `OrganizationId`; every query filters on the value resolved from the JWT.
- **Core AI instructions are platform-owned.** Tenants supply data only; the safety rules cannot be edited by them.
- **Nothing is hard deleted.** Tenant data uses `IsDeleted` soft delete; audit rows are append-only.

Two directions of traffic

Understanding the integration means separating them. **Outbound:** we push configuration to the voice provider (agent, prompt, tools, knowledge base). **Inbound:** the provider calls us during and after calls (tool functions, webhooks). They authenticate differently — see sections 11–13.

02 Technology stack

Layer	Choice	Notes
API	ASP.NET Core 8 Web API	Controllers, async throughout
Data access	Dapper 2.1	Hand-written SQL; no ORM change-tracking overhead
Database	SQL Server (AiDB)	Auto-created and migrated at startup
Auth	JWT Bearer + refresh tokens	BCrypt password hashing
Voice	Retell AI	STT, LLM orchestration, TTS, telephony
Frontend	React 18, Vite 6	React Router, Zustand, TanStack Query
Styling	Tailwind CSS v4	Design tokens in <code>index.css</code> <code>@theme</code>
Animation	Framer Motion	Card entry, mobile drawer
Docs	Swagger / OpenAPI	<code>/swagger</code>

Not yet wired

Redis (rate limiting is in-memory), SMTP delivery, MFA, Application Insights. Interfaces and configuration keys exist; implementations are stubs. See section 20.

03 Repository layout

```
ai-voice-agent/
├─ backend/AiReceptionist.Api/
│  │ Program.cs           // DI, auth, CORS, rate limits, pipeline
│  │ Common/
│  │ │ ApiResponse.cs     // standard envelope + PagedResult
│  │ │ TenantProvider.cs  // resolves OrganizationId from JWT
│  │ │ PhoneUtil.cs       // phone canonicalisation
│  │ │ TenantTime.cs      // tz conversion + BusinessHours parser
│  │ │ SqlUtil.cs         // LIKE wildcard escaping
│  │ │ RetellSignature.cs  // webhook HMAC + tool-call token
│  │ └─ Controllers/      // 12 controllers – see section 08
│  │ └─ Data/
│  │ │ └─ DbConnectionFactory.cs
│  │ │ └─ DbInitializer.cs // creates AiDB, schema, seed, migrations
│  │ │ └─ Repositories/    // 10 repositories
│  │ └─ Domain/Entities.cs // entities + Roles constants
│  └─ Middleware/ExceptionMiddleware.cs
├─ Services/
│  │ └─ TokenService.cs
│  │ └─ PromptBuilderService.cs // prompt / knowledge split
│  │ └─ IndustryTemplates.cs    // per-trade behaviour
│  │ └─ RetellService.cs        // provider sync + call backfill
│  │ └─ RetellSyncWorker.cs     // background queue consumer
├─ frontend/src/
│  │ └─ api/client.js           // axios + refresh interceptor
│  │ └─ store/auth.js           // Zustand persisted session
│  │ └─ components/{AppLayout,ui}.jsx
│  │ └─ pages/                  // 8 pages
└─ database/                   // standalone SQL scripts
```

04 Multi-tenancy model

One row in `Organizations` is one tenant. Isolation is enforced at three layers.

1 — Claim

`TokenService` writes an `orgId` claim into every access token. `TenantProvider` reads it per request and throws `UnauthorizedAccessException` if absent.

```
public int OrganizationId =>
    int.TryParse(Principal?.FindFirstValue("orgId"), out var id)
        ? id : throw new UnauthorizedAccessException("Missing organization claim.");
```

2 — Query

Every repository method takes `orgId` and includes it in the `WHERE` clause. There is no code path that reads a tenant table without it.

3 — Schema

Every tenant-owned table has a non-nullable `OrganizationId` foreign key.

Rule for new code

A repository method that touches a tenant table and does *not* accept `orgId` is a bug. Controllers must pass `_tenant.OrganizationId` — never a value from the request body, which a client could forge.

Inbound provider traffic

Tool calls and webhooks arrive without a JWT, so the tenant is resolved differently:

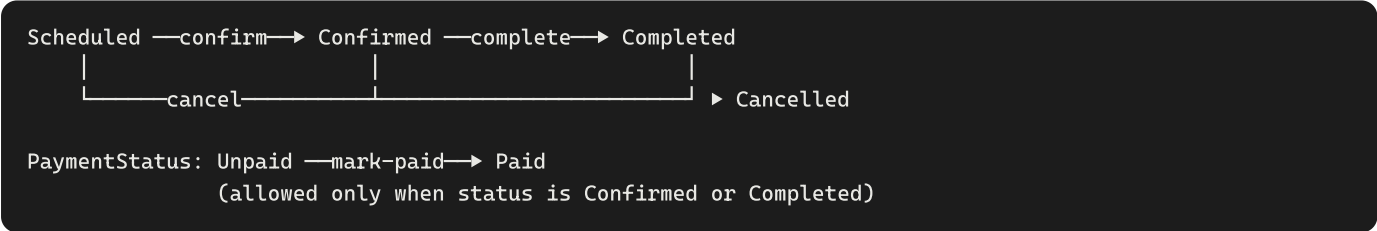
- **Tool calls** — `orgId` is a path segment in the URL registered with the provider, protected by a per-tenant token (section 12).
- **Webhooks** — resolved by looking up the payload's `agent_id` against `AgentConfig.RetellAgentId` (section 13).

05 Database schema

Twelve tables. `DbInitializer` creates the database, applies schema and additive migrations, and seeds demo data on an empty database — all idempotent, run at startup.

Table	Purpose & key columns
Organizations	Tenant root. <code>Industry</code> drives AI behaviour; <code>BusinessHoursJson</code> , <code>Timezone</code> , <code>MaxConcurrentAppointments</code> , <code>ProductsEnabled</code> , <code>Currency</code>
Users	<code>PasswordHash</code> (BCrypt), <code>Role</code> ; unique filtered index on <code>Email</code> where not deleted
RefreshTokens	Rotating tokens with <code>ExpiresAt</code> , <code>Revoked</code>
Customers	<code>Phone</code> canonicalised; <code>FirstVisit</code> , <code>LastVisit</code> , <code>TotalVisits</code> ; index on (<code>OrganizationId</code> , <code>Phone</code>)
Services	<code>DurationMinutes</code> , <code>MinPrice</code> / <code>MaxPrice</code> , <code>IsEmergency</code> , <code>IsAvailable</code>
Products	<code>Sku</code> , <code>Category</code> , <code>Price</code> , <code>Quantity</code>
Appointments	UTC <code>StartAt</code> / <code>EndAt</code> , <code>Status</code> , <code>PaymentStatus</code> , <code>Amount</code> , <code>ServiceAddress</code> , <code>IsEmergency</code> ; covering index on (<code>OrganizationId</code> , <code>StartAt</code>)
CallLogs	<code>RetellCallId</code> (idempotency key), <code>Transcript</code> , <code>RecordingUrl</code> , <code>Summary</code> , <code>DurationSeconds</code>
KnowledgeBase	<code>Category</code> ∈ BusinessInfo · FAQ · Policy · EmergencyRule · Custom
AgentConfig	One per tenant (unique <code>OrganizationId</code>). Voice, language, greeting, transfer number, and provider ids: <code>RetellAgentId</code> , <code>RetellLlmId</code> , <code>RetellKnowledgeBaseId</code> , <code>LastSyncedAt</code>
TimelineEvents	Per-customer activity feed; <code>Source</code> ∈ AI · User · System
AuditLogs	Append-only; not deletable by tenant users

Appointment status machine



Transitioning to `Completed` also increments the customer's `TotalVisits` and updates `FirstVisit` / `LastVisit` in the same statement.

06 Authentication & authorization

Token flow

- `POST /auth/login` verifies the BCrypt hash and issues a short-lived access token (15 min) plus a refresh token (7 days, persisted).
- The SPA stores both in Zustand (persisted to localStorage) and attaches the access token as a Bearer header via an axios interceptor.
- On `401`, the interceptor calls `POST /auth/refresh` once, rotates both tokens, and replays the original request. Concurrent 401s share one refresh promise.
- Refresh **rotates**: the presented token is revoked and a new one issued, so a stolen refresh token is single-use.

Access token claims

```
sub    = user id      orgId = tenant id    (custom claim)
email  = user email   role  = RBAC role    jti = token id
```

Roles

Role	Scope
SuperAdmin	Platform operator. Included in every permission set, plus exclusive access to provider sync/disconnect.
OrgAdmin	Full control of one tenant, including organisation settings
Manager	Catalogue, knowledge base, agent settings, operations
Receptionist	Appointments and customers
ReadOnly	GET only

Two composite constants avoid drift across controllers:

```
Roles.Staff      = SuperAdmin, OrgAdmin, Manager, Receptionist // operations
Roles.Management = SuperAdmin, OrgAdmin, Manager               // configuration
```

Lesson learned

Roles were originally listed inline per endpoint and `SuperAdmin` was omitted, making the platform operator *less* privileged than a tenant admin — buttons silently returned `403`. Always use the constants for new endpoints.

07 Request pipeline

```
ExceptionMiddleware // maps unhandled → 500 envelope with traceId
→ Swagger
→ ResponseCompression
→ CORS ("Frontend" policy, origins from config)
→ RateLimiter
→ Authentication → Authorization
→ MapControllers + /health
```

Response envelope

Every endpoint returns the same shape (SRS §21):

```
{
  "success": true,
  "message": "Appointment created successfully.",
  "data": { ... },      // or PagedResult<T>
  "errors": [],
  "timestamp": "2026-07-25T09:14:22Z",
  "traceId": "0HN7GK..." // correlation id, set on failures
}
```

Paged endpoints wrap data as { items, page, pageSize, totalCount, totalPages }.

Rate limiting

Partition	Limit
Provider traffic (/ai/tools/*, /webhooks/*)	3,000 / minute (shared bucket)
All other traffic (per IP)	200 / minute
login policy	10 / minute

Provider traffic gets its own generous bucket because it originates from a small number of egress IPs; sharing the per-IP bucket would throttle live calls mid-sentence.

08 Internal API reference

Base path `/api/v1`. All endpoints require a Bearer token unless noted. S = Roles.Staff, M = Roles.Management, SA = SuperAdmin, — = any authenticated role

Authentication — anonymous

Endpoint	Purpose
POST <code>/auth/login</code>	Issue access + refresh tokens
POST <code>/auth/refresh</code>	Rotate token pair
POST <code>/auth/logout</code>	Revoke a refresh token

Dashboard

Endpoint	Purpose
GET <code>/dashboard/stats</code> —	Today/upcoming/cancelled, missed calls, revenue, outstanding, AI booking rate, 12-month series. "Today" computed in tenant timezone.
GET <code>/dashboard/counts</code> —	Sidebar badges: appointments awaiting confirmation, missed calls, customers

Appointments

Endpoint	Purpose
GET <code>/appointments</code> —	Filter by <code>from</code> , <code>to</code> , <code>status</code> ; paged
GET <code>/appointments/{id}</code> —	Single, with joined customer/service/staff names
POST <code>/appointments</code> S	Create; <code>409</code> if the slot is full
POST <code>/appointments/{id}/confirm</code> S	Scheduled → Confirmed
POST <code>/appointments/{id}/mark-paid</code> S	Unpaid → Paid; <code>400</code> unless Confirmed/Completed
POST <code>/appointments/{id}/complete</code> S	→ Completed; updates visit history
POST <code>/appointments/{id}/cancel</code> S	→ Cancelled; frees the slot
POST <code>/appointments/{id}/reschedule</code> S	Move; <code>409</code> on conflict

Customers

Endpoint	Purpose
GET <code>/customers</code> —	<code>search</code> across name/phone/email; paged
GET <code>/customers/{id}</code> —	Single record
GET <code>/customers/{id}/timeline</code> —	Activity feed, newest first
POST <code>/customers</code> S	Create (phone canonicalised)
PUT <code>/customers/{id}</code> S	Update
DEL <code>/customers/{id}</code> M	Soft delete

Catalogue & knowledge

Endpoint		Purpose
GET POST PUT DEL /services	M	Service CRUD; writes trigger agent re-sync
GET POST PUT DEL /products	M	Product CRUD; writes trigger agent re-sync
GET POST PUT DEL /knowledge-base	M	Knowledge CRUD; writes trigger agent re-sync
GET /knowledge-base/final-prompt	—	Read-only preview: system prompt, its character count, and the knowledge documents with sizes

Calls & settings

Endpoint		Purpose
GET /calls · /calls/{id}	—	Call log with transcript, recording, summary
POST /calls/sync	—	Reconcile missing calls from the provider (section 13)
GET PUT /settings/organization	M	Business profile, hours, timezone, concurrency
GET PUT /settings/agent	M	Voice, language, greeting, transfer number. PUT preserves provider ids.
GET /retell/status	—	Connection state, agent/LLM/KB ids, last sync
POST /retell/sync	SA	Create or update the provider agent
POST /retell/disconnect	SA	Detach stored provider ids

09 Prompt architecture

`PromptBuilderService` splits tenant content by *how the model needs it*. This is a cost decision as much as a quality one: prompt tokens are billed on every conversational turn, knowledge-base content is retrieved only when relevant.

System prompt — every turn	Knowledge base — on demand
Role & personality	Services catalogue & pricing
Critical rules	Product catalogue
Industry rules	FAQs
Conversation flow	Policies
Business identity	Long-form business info
Business hours	Custom reference notes
Emergency rules	
Tool usage guide	

Emergency rules stay in the prompt deliberately

Retrieval is probabilistic — a document may not be fetched. Safety instructions ("tell the caller to shut off the water main", "advise them to leave the property") must be present on every turn, so they are never moved to the knowledge base.

Interface

```
Task<string> BuildSystemPromptAsync(int orgId);
Task<List<KnowledgeDocument>> BuildKnowledgeDocumentsAsync(int orgId);

public record KnowledgeDocument(string Title, string Text);
```

Composition order

```
Role line ("You are a plumbing company receptionist for X, a Y business")
+ Core rules      // platform-owned constant, never tenant-editable
+ Industry rules  // from IndustryTemplates
+ Conversation flow
+ Business details (+ field-service note when applicable)
+ Business hours (raw JSON)
+ Emergency rules (KnowledgeBase category = EmergencyRule)
+ Tool usage guide
```

The core rules constant encodes the non-negotiables: introduce as AI, never invent prices or availability, always use tools for live data, transfer when uncertain, confirm before mutating, one question at a time.

Observed sizes on the demo tenant: system prompt ≈ 2,960 characters; knowledge documents ≈ 1,340 characters across five documents. The gap widens sharply for tenants with large catalogues — which is the point.

10 Industry templates

`IndustryTemplates.Resolve(industry)` maps the free-text `Organizations.Industry` to an `IndustryProfile`, falling back to a generic premises-based receptionist. Matching is substring-based with a synonym pass ("plumber" → plumbing, "attorney" → legal, "clinic" → medical).

```
public record IndustryProfile(  
    string Role, // "a plumbing company receptionist"  
    bool IsFieldService, // technician travels to the caller  
    string CustomerNoun, // customer | patient | client  
    string[] ConversationFlow,  
    string[] ExtraRules);
```

The critical axis: field service vs premises

Aspect	Premises-based	Field service
Examples	Clinics, salons, workshops, law firms	Plumbing, electrical, HVAC, locksmith, cleaning
Flow length	8 steps	10 steps
Service address	Not collected	Mandatory — enforced in the tool schema and server-side
Urgency triage	Not prompted	Listens for leak / no power / no heat / lockout
Pricing language	Quoted range	Range, plus "final price depends on what the technician finds"

Eleven profiles ship today: medical, dental, plumbing, electrical, HVAC, locksmith, cleaning, automotive, salon, legal, consulting.

Adding an industry

Add an entry to the `Profiles` dictionary with its role string, field-service flag, customer noun and safety rules. No other code changes — the prompt, tool schema and required-field list all derive from the profile.

11 Voice provider integration (outbound)

`RetellService.SyncAgentAsync(orgId)` pushes a tenant's configuration to Retell AI. It is guarded by a per-organization `SemaphoreSlim` so the manual button and the background worker can never create duplicate resources.

Provider endpoints called

Endpoint	Used for
POST <code>/create-retell-llm</code>	First sync: prompt + tools + knowledge base ids
PATCH <code>/update-retell-llm/{llm_id}</code>	Subsequent syncs
POST <code>/create-agent</code>	First sync: voice, language, response engine, webhook
PATCH <code>/update-agent/{agent_id}</code>	Subsequent syncs
POST <code>/create-knowledge-base</code>	Multipart upload of reference documents
DEL <code>/delete-knowledge-base/{kb_id}</code>	Remove the superseded knowledge base
POST <code>/v2/list-calls</code>	Call reconciliation (section 13)

Authenticated with `Authorization: Bearer {Retell:ApiKey}`. Provider API calls are not separately billed; usage is billed per call minute.

Sync sequence

- **Guards** — API key configured; `PublicBaseUrl` must not be localhost (the provider rejects unreachable tool/webhook URLs).
- **Knowledge base** — build documents, upload as a new KB, capture its id.
- **LLM** — POST or PATCH with `general_prompt`, `begin_message`, `general_tools`, `knowledge_base_ids`.
- **Persist immediately** — write ids to `AgentConfig` before doing anything else, so a later failure cannot orphan a created resource.
- **Delete the previous KB** — only now, once the LLM no longer references it.
- **Agent** — POST or PATCH with voice, language, `response_engine = { type: "retell-llm", llm_id }` and `webhook_url`.
- **Persist** agent id and `LastSyncedAt`.

Ordering is a correctness requirement, not style

Create-then-swap-then-delete guarantees the agent always references a live knowledge base. Deleting first would leave a window where the agent points at nothing. Persisting ids immediately after creation prevents orphaned billable resources if the process dies mid-sync.

Provider-specific gotchas

- **E.164 only** — transfer numbers must be `+15550100199`, no spaces. `PhoneUtil.Normalize` is applied before sending. Fictional numbers (555 area code) are rejected outright.
- **transfer_call requires transfer_option** — omitting it fails schema validation.
- **Voice ids are provider-prefixed** — `openai-Nova`, `11labs-Adrian`. `MapVoice` translates legacy bare names.
- **Knowledge bases are immutable once indexed** — content changes mean creating a new one and swapping.

12 Live-call tool endpoints (inbound)

During a call the model invokes HTTP functions registered on the LLM. These are what make the AI factual — every price, slot and booking round-trips to AiDB.

Tool	Required arguments	Behaviour
<code>identify_customer</code>	phone	Phone → email → name lookup; returns visit count
<code>check_availability</code>	date	Slots honouring business hours, capacity and past-time exclusion
<code>book_appointment</code>	name, phone, service, start_time (+ service_address for field service)	Creates customer if new, atomic capacity-checked insert, timeline event
<code>cancel_appointment</code>	phone, name	Name verification; disambiguates by date/service
<code>reschedule_appointment</code>	phone, name, new_time	Conflict-checked move
<code>check_appointment</code>	phone	Next booking with status and payment status
<code>quote_service</code>	service	Real price range and duration
<code>quote_product</code>	product	Real price and stock

Plus provider built-ins `end_call` and `transfer_call`.

Authentication — the important part

Tool calls are NOT signed by the provider

Only webhooks carry `X-Retell-Signature`. Custom function calls are plain POSTs. An early implementation validated a signature on tool calls and returned `401` to every one of them — calls failed mid-conversation with "a technical error".

Because we generate the tool URLs during sync, the secret travels in the URL:

```
token = HMACSHA256(apiKey, $"tools:{orgId}") → hex, first 32 chars  
  
https://host/api/v1/ai/tools/1/book_appointment?k=36e98a8ba82d...
```

It is unguessable without the API key, scoped per tenant, and needs no storage. `RetellSignature.AcceptToolCall` accepts a valid token or a valid signature, and rejections log which credential was missing.

Response contract

Return JSON whose `result` string is written **as an instruction to the model**, not as UI copy — the model reads it and acts on it:

```
{ "result": "That time was just taken. Use check_availability to offer alternatives." }  
{ "result": "A service address is required before booking. Ask for the full street  
address, city, and any access details such as apartment or gate code." }
```

This is a server-side guardrail: even if the model calls a tool prematurely, the response steers it back on track.

13 Webhooks & call reconciliation

Webhook — `POST /api/v1/webhooks/retell`

Anonymous, verified by `X-Retell-Signature` (HMAC-SHA256 of the raw body keyed with the API key). The raw stream is read before deserialisation because the signature covers the exact bytes.

```
{ "event": "call_ended",
  "call": { "call_id", "agent_id", "from_number", "transcript",
            "recording_url", "start_timestamp", "end_timestamp",
            "disconnection_reason", "call_analysis": { "call_summary" } } }
```

- `call_ended` → store the call log, match the customer by phone, write an `AiCall` timeline event.
- `call_analyzed` → attach the summary; falls back to storing the call if the earlier event was missed.
- Tenant resolved via `agent_id` → `AgentConfig`. Unknown agents are acknowledged with `200` so the provider stops retrying.
- Duplicate deliveries are suppressed by checking `RetellCallId` first.

`disconnection_reason` maps to status: `dial_no_answer` / `dial_busy` / `dial_failed` → *Missed*; `call_transfer` → *Transferred*; otherwise *Completed*.

Reconciliation — why webhooks alone are not enough

Webhooks are fire-and-forget. If the API or tunnel is down when a call ends, that history is lost permanently.

`BackfillCallsAsync` pulls the last 100 calls for the tenant's agent from `POST /v2/list-calls` and inserts any whose `RetellCallId` is not already stored.

It runs **automatically on API startup** for every connected tenant, and on demand via `POST /calls/sync`. Idempotent, so it is safe to run repeatedly.

Web test calls have no caller ID; `from_number` is stored as `web-call` and cannot be auto-linked to a customer.

14 Background workers

`RetellSyncWorker` is a `BackgroundService` consuming an unbounded `Channel<int>` of organization ids.

```
public interface IRetellSyncQueue {
    void Enqueue(int orgId);
    ChannelReader<int> Reader { get; }
}
```

Producers

Controllers call `_retellSync.Enqueue(orgId)` after any mutation that changes what the AI should know: services, products, knowledge base entries, organization settings, agent settings.

Consumer behaviour

- Skips tenants with no `RetellAgentId` — the worker never creates provider resources implicitly; that is an explicit operator action.
- On startup, enqueues every connected tenant (so a changed public URL propagates) and runs call backfill for each.
- Exceptions are logged and swallowed per item; one bad tenant cannot stop the loop.

Scaling note

The queue is in-process and non-durable — enqueued work is lost on restart, mitigated by the startup re-sync. For horizontal scaling this should move to a distributed queue so the work is executed exactly once across instances.

15 Security model

Control	Implementation
Password storage	BCrypt with per-password salt
Access tokens	HMAC-SHA256 JWT, 15-minute lifetime, 30s clock skew
Refresh tokens	512-bit CSPRNG, persisted, rotated and revoked on use
Tenant isolation	<code>orgId</code> claim + mandatory query predicate
RBAC	<code>[Authorize(Roles = ...)]</code> via shared constants
Webhook authenticity	HMAC-SHA256 over raw body, fixed-time comparison
Tool-call authenticity	Per-tenant derived token in the registered URL
SQL injection	Parameterised Dapper throughout; LIKE wildcards escaped
Rate limiting	Partitioned fixed windows, separate login policy
CORS	Explicit origin allow-list from configuration
Audit trail	Append-only <code>AuditLogs</code> ; no tenant-facing delete path
Secrets	<code>appsettings.Local.json</code> (git-ignored) or environment variables

Secrets are layered: `appsettings.json` → `appsettings.Local.json` → environment. `JWT__SECRET` is read from the environment in preference to config.

Outstanding security gaps

- **No encryption at rest.** SRS §20 requires AES-256 for customer data, phone numbers, emails and recordings. Everything is currently plaintext.
- **MFA not implemented** — the `MfaEnabled` column exists, unused.
- **No email verification or password reset flow.**
- **Caller verification is weak by design** — cancel/reschedule check a name token against the record, which is a deterrent, not authentication.

16 Correctness & edge cases

Non-obvious behaviours that exist because the naive implementation was wrong.

Concurrent booking

Check-then-insert races when two calls are live simultaneously. Booking is a single statement inside a transaction using range locks:

```
INSERT INTO Appointments (...) OUTPUT INSERTED.Id
SELECT @... WHERE
  (SELECT COUNT(*) FROM Appointments WITH (UPDLOCK, HOLDLOCK)
   WHERE OrganizationId=@o AND IsDeleted=0
    AND Status NOT IN ('Cancelled','Missed')
    AND StartAt < @EndAt AND EndAt > @StartAt)
< (SELECT ISNULL(MaxConcurrentAppointments,1) FROM Organizations WHERE Id=@o)
```

Returns `null` when the slot is full — callers translate that to `409` or a spoken "that time was just taken".

Time zones

Appointments are stored in **UTC**; callers and the AI speak tenant-local time. `TenantTime` converts at the boundary using `Organizations.Timezone`. Dashboard "today" is computed from tenant-local midnight, not `GETUTCDATE()`.

Phone numbers

`PhoneUtil.Normalize` reduces to `+` plus digits. Applied on every write and lookup, with a trailing-nine-digit fallback so a missing country code still matches. Verified: `+1 (555) 010-0888`, `555-010-0888` and `5550100888` resolve to one customer.

Business hours

`BusinessHours.For(json, dayOfWeek)` parses `{"mon-fri":"09:00-17:00","sat":"10:00-14:00","sun":"closed"}`, supporting single days, ranges and wrapped ranges. A configured JSON that omits a day means *closed*; absent configuration falls back to 09:00–17:00 daily.

Case	Handling
Booking in the past	Rejected against tenant-local now
Booking outside hours	Rejected with the actual window in the message
Closed day	"The business is closed on Sundays. Offer another day."
Multiple matching appointments	Model is told to ask which one, with the list
Duplicate webhook	<code>RetellCallId</code> existence check
LIKE wildcards in caller speech	<code>SqlUtil.EscapeLike</code>
Concurrent sync	Per-org semaphore
Rotating tunnel URL	Startup re-sync of all connected tenants

17 Frontend architecture

Concern	Approach
Routing	React Router; <code>RequireAuth</code> wraps the authenticated shell
Server state	TanStack Query — 30s stale time, no refetch on focus
Session	Zustand with <code>persist</code> (localStorage)
HTTP	axios, relative <code>/api/v1</code> base so any tunnel works
Layout	<code>h-dvh</code> shell; only <code><main></code> scrolls
Mobile	Sidebar becomes an animated drawer below <code>md</code>

Design tokens

Defined once in `index.css` under Tailwind v4's `@theme`: page `#b7d5d4`, panel `#f2f2ee`, card white, ink `#1c1c1c`, accent `#f4ec7a`, plus lavender/mint/cream chips and sky/leaf chart colours. Cards use a 24px radius, buttons are full pills.

Cache invalidation

Mutations invalidate the query keys they affect — for example confirming an appointment invalidates `['appointments']`, `['dashboard']` and `['sidebar-counts']`, so badges and figures update without a reload.

Surface failures

Mutations must render their errors. Silent `catch` blocks previously made `403` responses look like dead buttons — the appointment actions now show the server message and a per-button busy state.

18 Configuration reference

Key	Purpose
ConnectionStrings:AiDB	Application database
ConnectionStrings:Master	Used once to <code>CREATE DATABASE</code>
Jwt:Secret / JWT__SECRET	Signing key; environment wins
Jwt:AccessTokenMinutes	Default 15
Jwt:RefreshTokenDays	Default 7
Cors:AllowedOrigins	SPA origin allow-list
Retell:ApiKey	Bearer credential <i>and</i> HMAC key for inbound verification
Retell:ApiBaseUrl	Default <code>https://api.retellai.com</code>
Retell:WebhookBaseUrl	Public HTTPS base for tool and webhook URLs. Must not be localhost.
Retell:DefaultVoiceId	Fallback voice
Retell:VerifySignature	Set false only for local testing

Never commit secrets

`appsettings.Local.json` is git-ignored and holds the API key locally. In production use environment variables or a secret store. Rotate any key that has been pasted into a chat, ticket or log.

19 Local setup & operations

First run

```
# Backend - creates AiDB, schema and demo data automatically
cd backend/AiReceptionist.Api
dotnet run --urls http://localhost:5200

# Frontend - proxies /api to :5200
cd frontend
npm install
npm run dev
```

Swagger at `http://localhost:5200/swagger` (plain **http** — https is not bound). Health at `/health`.

Seeded accounts

Email	Password	Role
<code>superadmin@demo.com</code>	<code>Super123!</code>	SuperAdmin
<code>admin@demo.com</code>	<code>Admin123!</code>	OrgAdmin

Exposing the API for voice integration

The provider must reach this API over public HTTPS.

```
ngrok http 5200 # then set Retell:WebhookBaseUrl to the https URL
```

Restart the API afterwards — the startup worker re-syncs every connected tenant so the agent picks up the new URLs. Free tunnel URLs rotate on restart, so this is a routine step.

Troubleshooting

Symptom	Cause / fix
Sync refuses with a localhost message	<code>WebhookBaseUrl</code> unset or still localhost
Tool calls return 401	URL missing <code>?k=</code> ; re-sync to regenerate
Transfer number rejected	Not valid E.164, or a fictional 555 number
Calls missing from the log	Webhook missed while down — run <code>POST /calls/sync</code>
Buttons appear inert	Role lacks permission; check the error banner and role constants
Swagger unreachable	Using https, or hitting the SPA tunnel rather than :5200
SSMS cannot connect	Tick "Trust server certificate"; server name uses one backslash

20 Known gaps & roadmap

Not implemented

Area	Status
Encryption at rest	Required by SRS §20; not started
Email (SMTP)	Config keys exist; no sender, no templates, no reminders
Redis	Rate limiting in-memory; no distributed cache
MFA, email verification, password reset	Not started
Onboarding wizard	SRS §18; settings pages exist instead
Configurable permissions	Roles are fixed constants, not per-tenant
Staff scheduling	<code>StaffUserId</code> exists; no rosters or skill matching
Observability	Structured logs only; no Application Insights or metrics

Recommended next steps

1. **Encryption at rest** for customer PII — the largest compliance gap.
2. **Durable job queue** to replace the in-process channel before scaling out.
3. **Per-tenant usage metering** — sum `CallLogs.DurationSeconds` by organization; this is the basis of both billing and cost control.
4. **Knowledge-base thresholding** — provider knowledge bases carry a monthly fee each; keep small catalogues inline in the prompt and only create a KB above a size threshold.
5. **Field-service depth** — travel-time buffers, call-out fees, technician skill matching.
6. **Email notifications** — confirmations and reminders are the highest-value missing customer-facing feature.

Scale considerations

The SRS targets 10,000+ organizations. At that scale: the sync queue must be distributed; per-tenant provider knowledge bases become a material recurring cost; dashboard aggregates should be cached rather than computed per request; and `CallLogs.Transcript` growth warrants partitioning or archival.